

Final Project Report

Fady Youssef

CPSC 321, Fall 2024

Video Link: <https://youtu.be/VfrstyZUI7k>

1 Summary

For this project, I decided to build a system called "Apex Fleet Manager." It's basically a database for a car rental company that brings everything together in one place. It handles three main things: keeping track of cars across different cities, managing the whole rental process (like booking and returning cars), and helping managers check how the business is doing. The main goal was to stop using messy records and switch to a proper database that keeps data safe and makes it easy to find what you need.

2 Use Cases

2.1 Customer Registration

Type: Creating data New customers can create an account. The system asks for their details like their license number and address, and then it gives them a unique Customer ID they use to log in.

2.2 Vehicle Search and Discovery

Type: Searching for data Users can search the database to find cars they want to rent. They can filter by city and car type (like SUV or Economy) to see only the cars that are actually available right now.

2.3 Reservation Booking

Type: Creating data When a user picks a car, the system makes a reservation. It saves the booking, handles the payment, and immediately marks the car as "Rented" so nobody else can book it at the same time.

2.4 Profile Management

Type: Updating data Customers can check their profile and update their phone number or address if they move.

2.5 Fleet Return Processing

Type: Updating data When a customer brings a car back, the staff uses this feature to check it in. It updates the car's mileage in the database and changes the status back to "Available" so it's ready for the next person.

2.6 Maintenance Logging

Type: Creating and Updating data If a car is damaged or needs an oil change, staff can flag it. This creates a maintenance record with the cost and marks the car as "Maintenance" so customers can't rent it.

2.7 Manager Analytics (Revenue Report)

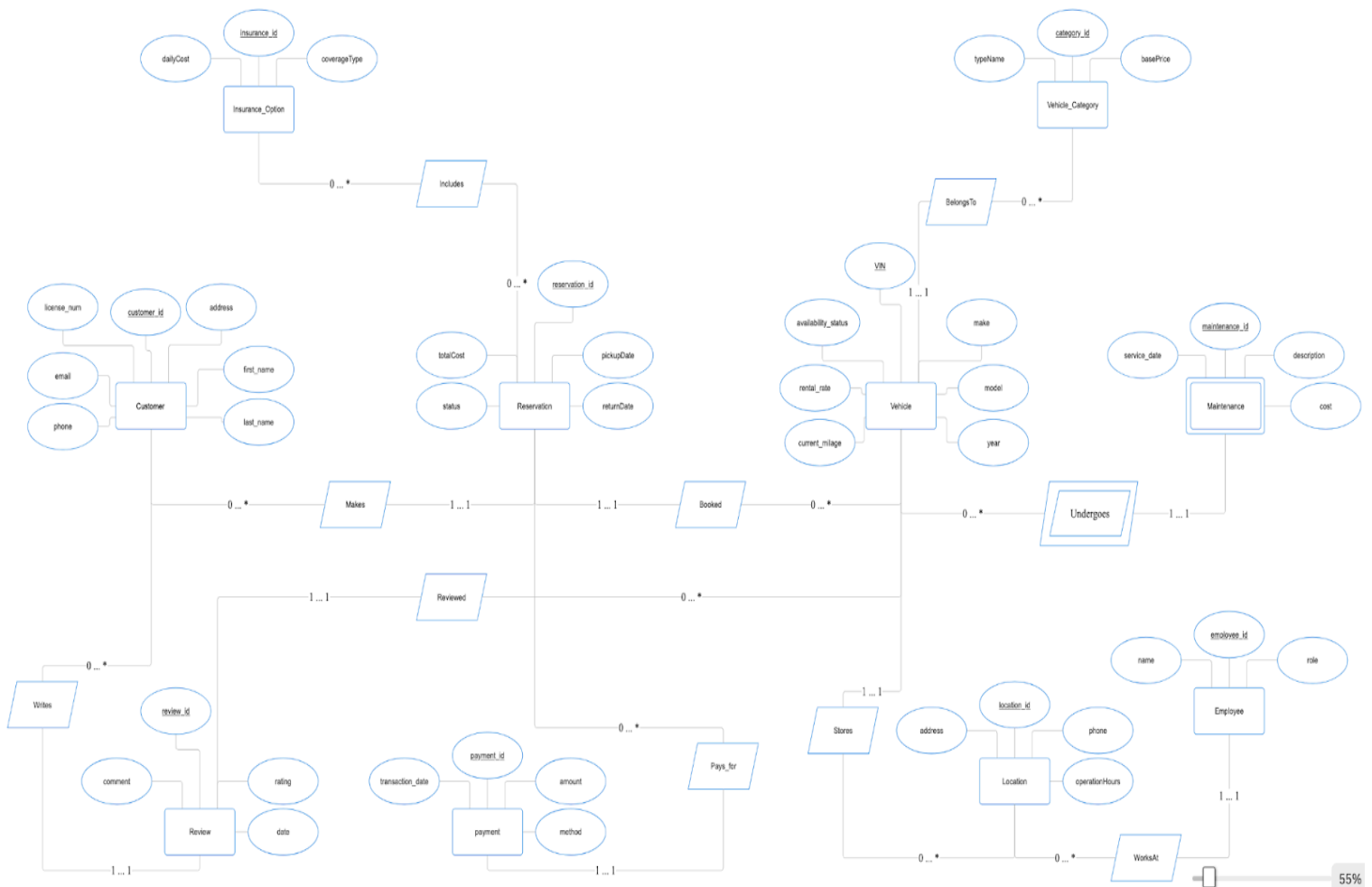
Type: Analyzing data Managers can run a report to see how much money each location is making. It groups the sales by city so they can see which branch is doing the best.

2.8 VIP Customer Tracking

Type: Analyzing data This finds the best customers. It adds up all the money a customer has ever spent and shows a list of the "VIPs" who have spent more than \$2,000.

3 Logical Design

3.1 Entity-Relationship Diagram



My ER diagram follows the 3NF rules to keep things organized. The main parts are **Vehicles** and **Customers**, and they connect through **Reservations**. I put things like **Locations** and **Vehicle_Categories** in their own tables so I didn't have to keep repeating the same text over and over for every car.

3.2 Relational Schema

- **Locations** (location_id, address, phone, operating_hours)
- **Vehicle_Categories** (category_id, type_name, base_price)
- **Insurance_Options** (insurance_id, coverage_type, daily_cost)
- **Customers** (customer_id, name, phone, email, license_num, address)
- **Vehicles** (vin, make, model, year, current_mileage, availability_status, location_id, category_id)
- **Reservations** (reservation_id, customer_id, vin, pickup_date, return_date, total_cost, status)
- **Payments** (payment_id, reservation_id, amount, method, transaction_date)
- **Maintenance_Records** (maintenance_id, vin, service_date, description, cost)
- **Reviews** (review_id, customer_id, vin, rating, comment, review_date)

4 Use-Case SQL Statements

For each use case, give the corresponding SQL statements that you developed. Each SQL statement should support the use case. Be sure to describe each query (one sentence) and state the corresponding parameters (which you can denote as ? or %s, depending on how you implemented them in your applications). Organize this section like section 3 (each use case as a separate subsection).

4.1 Customer Registration

This query adds a new user to the table and returns their new ID.

```
INSERT INTO Customers (name, phone, email, license_num, address)
VALUES (%s, %s, %s, %s, %s)
RETURNING customer_id;
```

4.2 Vehicle Search and Discovery

This query joins three tables to show the user clear details (Make, Model, Price) for cars that match what they are looking for.

```
SELECT v.vin, v.make, v.model, v.year, c.base_price
FROM Vehicles v
JOIN Vehicle_Categories c ON v.category_id = c.category_id
WHERE v.location_id = %s AND v.category_id = %s AND v.availability_status = 'Available';
```

4.4 Profile Management

This lets a user change their contact info using their ID.

```
UPDATE Customers SET phone = %s, address = %s WHERE customer_id = %s;
```

4.5 Fleet Return Processing

This updates the mileage on the car and sets it free for the next customer.

```
UPDATE Vehicles SET current_mileage = %s, availability_status='Available'
WHERE vin=%s;
```

4.6 Maintenance Logging

This saves a record of the repair and makes sure the car isn't listed as available.

```
INSERT INTO Maintenance_Records (vin, service_date, description, cost)
VALUES (%s, CURRENT_DATE, %s, %s);
```

4.7 Manager Analytics (Revenue Report)

This calculates total sales for each branch. I used a CASE statement here to make sure branches with no sales show up as 0 instead of just being empty.

```
SELECT l.address,
       CASE
         WHEN SUM(p.amount) IS NULL THEN 0
         ELSE SUM(p.amount)
       END as total
FROM Locations l
LEFT JOIN Vehicles v ON l.location_id = v.location_id
LEFT JOIN Reservations r ON v.vin = r.vin
LEFT JOIN Payments p ON r.reservation_id = p.reservation_id
GROUP BY l.address ORDER BY total DESC;
```

4.8 VIP Customer Tracking

This query uses SUM to find the total spending and HAVING to filter for people who spent over \$2,000.

```
SELECT c.name, SUM(p.amount) as spent
FROM Customers c
JOIN Reservations r ON c.customer_id = r.customer_id
JOIN Payments p ON r.reservation_id = p.reservation_id
GROUP BY c.name HAVING SUM(p.amount) > 2000 ORDER BY spent DESC;
```

5 Applications

For the final project, I built a command-line app using Python 3.13 and PostgreSQL 16. I used `psycopg2` to connect them, which let me use parameterized queries to keep the database safe from SQL injection attacks. I also wrote a script called `seed_data.py` that automatically fills the database with over 60 records (like cars and customers) so the system is always ready to demo.

6 Conclusions

I managed to build a working database app that handles the main stuff a rental company needs, from booking cars to checking revenue. The system follows strict rules, like making sure you can't delete a car if it has a rental history. The biggest challenges I had were dealing with Foreign Key errors when trying to delete things, and getting Python dates to match up with PostgreSQL dates. If I had more time, I'd probably move this to a website using Flask so people could actually access it from their phones instead of just a terminal.